

Title: A Unified Executors Proposal for C++
Authors: Jared Hoberock, jhoberock@nvidia.com
Michael Garland, mgarland@nvidia.com
Chris Kohlhoff, chris@kohlhoff.com
Chris Mysen, mysen@google.com
Carter Edwards, hcedwar@sandia.gov
Other Contributors: Hans Boehm, hboehm@google.com
Gordon Brown, gordon@codeplay.com
Thomas Heller, thom.heller@gmail.com
Lee Howes, xrikcus@gmail.com
Hartmut Kaiser, hartmut.kaiser@gmail.com
Bryce Lebach, brycelebach@gmail.com
Gor Nishanov, gorn@microsoft.com
Thomas Rodgers, rodgert@twroddgers.com
Michael Wong, michael@codeplay.com
Document Number: P0443R0
Date: 2016-10-17
Reply-to: jhoberock@nvidia.com
Audience: SG1 - Concurrency and Parallelism, SG14 - Low Latency, LEWG

1 Introduction

This paper describes a programming model for *executors*, which are modular components for creating execution. Executors decouple control structures from the details of work creation and prevent multiplicative explosion inside control structure implementations. The model proposed by this paper represents what we think is the *minimal* functionality necessary to compose executors with existing standard control structures such as `std::async()` and parallel algorithms, as well as near-standards such as the functionality found in various technical specifications, including the Concurrency, Networking, and Parallelism TSes. While this paper's feature set is minimal, it will form the basis for future development of executor features which are out of the scope of a basic proposal.

Our executor programming model was guided by years of independent design work by various experts. This proposal is the result of harmonizing that work in collaboration with those experts for several months. In particular, our programming model unifies three separate executor design tracks aimed at disparate use cases:

1. Google's executor model for interfacing with thread pools [N4414](#),
2. Chris Kohlhoff's executor model for the Networking TS [N4370](#), and
3. NVIDIA's executor model for the Parallelism TS [P0058](#).

This unified executor proposal serves the use cases of those independent proposals with a single consistent programming model.

Executor categories. This proposal categorizes executor types in terms of requirements on those types. An executor type is a member of one or more executor categories if it provides member functions and types with the semantics that those categories require. These categories are used in generic interfaces to communicate the requirements on executors interoperating with them. Such interfaces are already present in the C++ Standard; for example, control structures like `std::async()`, `std::invoke()`, and the parallel algorithms library. Other control structures this proposal targets are found in the Concurrency, Networking, and Parallelism TSes.

Using executors with control structures. We expect that the primary way that most programmers will interact with executors is by using them as parameters to control structures. When used as a parameter to a control structure, an executor indicates "where" the execution created by the control structure should happen.

For example, a programmer may create an asynchronous task via `async()` by providing an executor:

```
auto my_task = ...;  
auto my_executor = ...;  
auto fut = async(my_executor, my_task);
```

In this example, the executor parameter provides `std::async()` with explicit requirements concerning how to create the work responsible for executing the task.

Similarly, a programmer may require that the work created by a parallel algorithm happen "on" an executor:

```
auto my_task = ...;
vector<int> vec = ...;
auto my_executor = ...;
for_each(execution::par.on(my_executor), vec.begin(), vec.end(), my_task);
```

Executor customization points. Executor categories require executor types to provide member functions with expected semantics. For example, the executor category `OneWayExecutor` requires an executor type to provide the member function `.execute(f)`, which may or may not block its caller pending completion of the function `f`. As another example, the executor category `TwoWayExecutor` requires an executor type to provide the member function `.async_execute(f)`, which returns a future object corresponding to the eventual completion of the function `f`'s invocation.

In non-generic contexts, clients of executors may create work by calling the member functions of executors directly:

```
template<class Function>
future<result_of_t<Function()>>
foo(const simple_two_way_executor& exec, Function f)
{
    return exec.async_execute(f);
}
```

However, directly calling executor member functions is impossible in generic contexts where the concrete type of the executor, and therefore the availability of specific member functions, is unknown. To serve these use cases, for each of these special executor member functions, we introduce an executor [customization point](#) in namespace `execution::`. These customization points adapt the given executor in such a way as to guarantee the execution semantics of the customization point even if it is not natively provided by the executor as a member function.

For example, the customization point `execution::async_execute()` allows `foo()` to compose with all executor types:

```
template<class Executor, class Function>
executor_future_t<Executor,result_of_t<Function()>>
foo(const Executor& exec, Function f)
{
    return execution::async_execute(exec, f);
}
```

These customization points allow higher-level control structures and "fancy" executors which adapt the behavior of more primitive executors to manipulate all types of executors uniformly.

Defining executors. Programmers may define their own executors by creating a type which satisfies the requirements of one or more executor categories. The following example creates a simple executor fulfilling the requirements of the `OneWayExecutor` category which logs a message before invoking a function:

```
class logging_context
{
public:
    void log(std::string msg);

    bool operator==(const logging_context& rhs) const noexcept
    {
        return this == &rhs;
    }
};

class logging_executor
{
public:
    logging_executor(logging_context& ctx) : context_(ctx) {}

    bool operator==(const logging_executor& rhs) const noexcept
    {
        return context() == rhs.context();
    }

    bool operator!=(const logging_executor& rhs) const noexcept
    {
        return !(*this == rhs);
    }

    const logging_context& context() const noexcept
    {
        return context_;
    }
}
```

```

template<class Function>
void execute(Function&& f) const
{
    context_.log("executing function");
    f();
}

private:
    mutable logging_context& context_;
};

```

Executors are also useful in insulating non-standard means of creating execution from the surrounding environment. The following example defines an executor fulfilling the requirements of the `BulkTwoWayExecutor` category which uses OpenMP language extensions to invoke a function a number of times in parallel:

```

class omp_executor
{
public:
    using execution_category = parallel_execution_tag;

    bool operator==(const omp_executor&) const noexcept
    {
        return true;
    }

    bool operator!=(const omp_executor&) const noexcept
    {
        return false;
    }

    const omp_executor& context() const noexcept
    {
        return *this;
    }

    template<class Function, class ResultFactory, class SharedFactory>
    auto bulk_sync_execute(Function f, size_t n,
                          ResultFactory result_factory,
                          SharedFactory shared_factory) const
    {
        auto result = result_factory();
        auto shared_arg = shared_factory();

        #pragma omp parallel for
        for(size_t i = 0; i < n; ++i)
        {
            f(i, result, shared_arg);
        }

        return result;
    }
};

```

1.1 Conceptual Elements

Instruction Stream: Code to be run in a form appropriate for the target execution architecture.

Execution Architecture: Denotes the target architecture for an instruction stream. The instruction stream defined by the *main* entry point and associated global object initialization instruction streams is the *host process* execution architecture. Other possible target execution architectures include attached accelerators such as GPU, remote procedure call (RPC), and database management system (DBMS) servers. The execution architecture may impose architecture-specific constraints and provides architecture-specific facilities for an instruction stream.

Execution Resource: An instance of an execution architecture that is capable of running an instruction stream targeting that architecture. Examples include a collection of `std::thread` within the host process that are bound to particular cores, GPU CUDA stream, an RPC server, a DBMS server. Execution resources typically have weak *locality* properties both with respect to one another and with respect to memory resources. For example, cores within a non-uniform memory access (NUMA) region are *more local* to each other than cores in different NUMA regions and hyperthreads within a single core are more local to each other than hyperthreads in different cores.

Execution Agent: An instruction stream is run by an execution agent on an execution resource. An execution agent may be *lightweight* in that its existence is only observable while the instruction stream is running. As such a lightweight execution agent may come into existence when the instruction stream starts running and cease to exist when the instruction stream ends.

Execution Context: The mapping of execution agents to execution resources.

Execution Function: The binding of an instruction stream to one or more execution agents. The instruction stream of a parallel algorithm may be bound to multiple execution agents that can run concurrently on an execution resource. An instruction stream's entry and return interface conforms to a specification defined by an execution function. An execution function targets a specific execution architecture.

Executor: Provides execution functions for running instruction streams on an particular, observable execution resource. A particular executor targets a particular execution architecture.

2 Proposed Wording

2.0.1 Header <execution> synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {
namespace execution {

    // Executor type traits:

    template<class T> struct is_one_way_executor;
    template<class T> struct is_host_based_one_way_executor;
    template<class T> struct is_non_blocking_one_way_executor;
    template<class T> struct is_bulk_one_way_executor;
    template<class T> struct is_two_way_executor;
    template<class T> struct is_bulk_two_way_executor;

    template<class T> constexpr bool is_one_way_executor_v = is_one_way_executor<T>::value;
    template<class T> constexpr bool is_host_based_one_way_executor_v = is_host_based_one_way_executor<T>::value;
    template<class T> constexpr bool is_non_blocking_one_way_executor_v = is_non_blocking_one_way_executor<T>::value;
    template<class T> constexpr bool is_bulk_one_way_executor_v = is_bulk_one_way_executor<T>::value;
    template<class T> constexpr bool is_two_way_executor_v = is_two_way_executor<T>::value;
    template<class T> constexpr bool is_bulk_two_way_executor_v = is_bulk_two_way_executor<T>::value;

    template<class Executor> struct executor_context;

    template<class Executor>
        using executor_context_t = typename executor_context<Executor>::type;

    template<class Executor, class T> struct executor_future;

    template<class Executor, class T>
        using executor_future_t = typename executor_future<Executor, T>::type;

    // Bulk executor traits:

    struct sequenced_execution_tag {};
    struct parallel_execution_tag {};
    struct unsequenced_execution_tag {};

    // TODO a future proposal can define this category
    // struct concurrent_execution_tag {};

    template<class Executor> struct executor_execution_category;

    template<class Executor>
        using executor_execution_category_t = typename executor_execution_category<Executor>::type;

    template<class Executor> struct executor_shape;

    template<class Executor>
        using executor_shape_t = typename executor_shape<Executor>::type;

    template<class Executor> struct executor_index;

    template<class Executor>
        using executor_index_t = typename executor_index<Executor>::type;

    // Executor customization points:

    template<class OneWayExecutor, class Function>
        void execute(const OneWayExecutor& exec, Function&& f);

    template<class NonBlockingOneWayExecutor, class Function>
```

```

void post(const NonBlockingOneWayExecutor& exec, Function&& f);

template<class NonBlockingOneWayExecutor, class Function>
void defer(const NonBlockingOneWayExecutor& exec, Function&& f);

template<class TwoWayExecutor, class Function>
result_of_t<decay_t<Function>()>
    sync_execute(const TwoWayExecutor& exec, Function&& f);
template<class OneWayExecutor, class Function>
result_of_t<decay_t<Function>()>
    sync_execute(const OneWayExecutor& exec, Function&& f);

template<class TwoWayExecutor, class Function>
executor_future_t<TwoWayExecutor, result_of_t<decay_t<Function>()>>
    async_execute(const TwoWayExecutor& exec, Function&& f);
template<class Executor, class Function>
std::future<decay_t<result_of_t<decay_t<Function>()>>
    async_execute(const Executor& exec, Function&& f);

template<class NonBlockingTwoWayExecutor, class Function>
executor_future_t<NonBlockingTwoWayExecutor, result_of_t<decay_t<Function>()>>
    async_post(const NonBlockingTwoWayExecutor& exec, Function&& f);
template<class NonBlockingOneWayExecutor, class Function>
std::future<decay_t<result_of_t<decay_t<Function>()>>
    async_post(const NonBlockingOneWayExecutor& exec, Function&& f);

template<class NonBlockingTwoWayExecutor, class Function>
executor_future_t<NonBlockingTwoWayExecutor, result_of_t<decay_t<Function>()>>
    async_defer(const NonBlockingTwoWayExecutor& exec, Function&& f);
template<class NonBlockingOneWayExecutor, class Function>
std::future<decay_t<result_of_t<decay_t<Function>()>>
    async_defer(const NonBlockingOneWayExecutor& exec, Function&& f);

template<class TwoWayExecutor, class Function, class Future>
executor_future_t<TwoWayExecutor, see-below>
    then_execute(const TwoWayExecutor& exec, Function&& f, Future& predecessor);
template<class OneWayExecutor, class Function, class Future>
executor_future_t<OneWayExecutor, see-below>
    then_execute(const OneWayExecutor& exec, Function&& f, Future& predecessor);

template<class BulkOneWayExecutor, class Function1, class Function2>
void bulk_execute(const BulkOneWayExecutor& exec, Function1 f,
    executor_shape_t<BulkOneWayExecutor> shape,
    Function2 shared_factory);
template<class OneWayExecutor, class Function1, class Function2>
void bulk_execute(const OneWayExecutor& exec, Function1 f,
    executor_shape_t<OneWayExecutor> shape,
    Function2 shared_factory);

template<class BulkTwoWayExecutor, class Function1, class Function2, class Function3>
result_of_t<Function2()>
    bulk_sync_execute(const BulkTwoWayExecutor& exec, Function1 f,
        executor_shape_t<BulkTwoWayExecutor> shape,
        Function2 result_factory, Function3 shared_factory);
template<class OneWayExecutor, class Function1, class Function2, class Function3>
result_of_t<Function2()>
    bulk_sync_execute(const OneWayExecutor& exec, Function1 f,
        executor_shape_t<OneWayExecutor> shape,
        Function2 result_factory, Function3 shared_factory);

template<class BulkTwoWayExecutor, class Function1, class Function2, class Function3>
executor_future_t<const BulkTwoWayExecutor, result_of_t<Function2()>>
    bulk_async_execute(const BulkTwoWayExecutor& exec, Function1 f,
        executor_shape_t<BulkTwoWayExecutor> shape,
        Function2 result_factory, Function3 shared_factory);
template<class OneWayExecutor, class Function1, class Function2, class Function3>
executor_future_t<const OneWayExecutor, result_of_t<Function2()>>
    bulk_async_execute(const OneWayExecutor& exec, Function1 f,
        executor_shape_t<OneWayExecutor> shape,
        Function2 result_factory, Function3 shared_factory);

template<class BulkTwoWayExecutor, class Function1, class Future, class Function2, class Function3>
executor_future_t<Executor, result_of_t<Function2()>>
    bulk_then_execute(const BulkTwoWayExecutor& exec, Function1 f,
        executor_shape_t<BulkTwoWayExecutor> shape,
        Future& predecessor,
        Function2 result_factory, Function3 shared_factory);
template<class OneWayExecutor, class Function1, class Future, class Function2, class Function3>
executor_future_t<OneWayExecutor, result_of_t<Function2()>>

```

```

        bulk_then_execute(const OneWayExecutor& exec, Function1 f,
                          executor_shape_t<OneWayExecutor> shape,
                          Future& predecessor,
                          Function2 result_factory, Function3 shared_factory);

// Executor work guard:

template <class Executor>
    class executor_work_guard;

// Polymorphic executor wrappers:

class one_way_executor;
class host_based_one_way_executor;
class non_blocking_one_way_executor;
class two_way_executor;
class non_blocking_two_way_executor;

} // namespace execution
} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

2.1 Requirements

2.1.1 Future requirements

A type `F` meets the future requirements for some value type `T` if `F` is... *Requirements to be defined. Futures must provide `get`, `wait`, `then`, etc.*

2.1.2 ProtoAllocator requirements

A type `A` meets the proto-allocator requirements if `A` is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where `U` is an object type. [Note: For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. --end note] No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

2.1.3 ExecutionContext requirements

A type meets the `ExecutionContext` requirements if it satisfies the `EqualityComparable` requirements (C++Std [equalitycomparable]). No comparison operator on these types shall exit via an exception.

2.1.4 BaseExecutor requirements

A type `X` meets the `BaseExecutor` requirements if it satisfies the requirements of `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `EqualityComparable` (C++Std [equalitycomparable]), as well as the additional requirements listed below.

No comparison operator, copy operation, move operation, swap operation, or member function context on these types shall exit via an exception.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

The destructor shall not block pending completion of the submitted function objects. [Note: The ability to wait for completion of submitted function objects may be provided by the associated execution context. --end note]

In the Table below, `x1` and `x2` denote (possibly const) values of type `X`, `mx1` denotes an xvalue of type `X`, and `u` denotes an identifier.

(Base executor requirements)	
Expression Type	Assertion/note/pre-/post-condition
	Shall not exit via an exception.
<code>X u(x1);</code>	<i>Post:</i> <code>u == x1</code> and <code>u.context() == x1.context()</code> .
	Shall not exit via an exception.
<code>X u(mx1);</code>	<i>Post:</i> <code>u</code> equals the prior value of <code>mx1</code> and <code>u.context()</code> equals the prior value of <code>mx1.context()</code> .

<code>x1 == x2</code>	<code>bool</code>	Returns <code>true</code> only if <code>x1</code> and <code>x2</code> can be interchanged with identical effects in any of the expressions defined in these type requirements (TODO and the other executor requirements defined in this Technical Specification). [Note: Returning <code>false</code> does not necessarily imply that the effects are not identical. --end note] <code>operator==</code> shall be reflexive, symmetric, and transitive, and shall not exit via an exception.
<code>x1 != x2</code>	<code>bool</code>	Same as <code>!(x1 == x2)</code> .
<code>x1.context()</code>	<code>E& OR const E&</code> where <code>E</code> is a type that satisfies the <code>ExecutionContext</code> requirements.	Shall not exit via an exception. The comparison operators and member functions defined in these requirements (TODO and the other executor requirements defined in this Technical Specification) shall not alter the reference returned by this function.

2.1.5 OneWayExecutor requirements

The `OneWayExecutor` requirements form the basis of the one-way executor concept taxonomy. This set of requirements specifies operations for creating execution agents that need not synchronize with the thread which created them.

A type `X` satisfies the `OneWayExecutor` requirements if it satisfies the `BaseExecutor` requirements, as well as the additional requirements listed below.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

In the Table below, `x` denotes a (possibly `const`) value of type `X`, and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

(One-way executor requirements)

Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
<code>x.execute(f)</code>		Creates a weakly parallel execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code> . May block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution.	<i>Synchronization:</i> The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code> .

2.1.6 HostBasedOneWayExecutor requirements

The `HostBasedOneWayExecutor` requirements form the basis of host-based executors in the one-way executor concept taxonomy. *TODO:* description of what host-based means, i.e. as if executed in a `std::thread`, but without the requirement for separate thread-local storage or a unique thread ID.

A type `X` satisfies the `HostBasedOneWayExecutor` requirements if it satisfies the `OneWayExecutor` requirements, as well as the additional requirements listed below.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

In the Table below, `x` denotes a (possibly `const`) value of type `X`, `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements, and `a` denotes a (possibly `const`) value of type `A` satisfying the `ProtoAllocator` requirements.

(Host-based one-way executor requirements)

Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
<code>x.execute(f)</code> <code>x.execute(f,a)</code>		Creates a parallel execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>execute</code> . May block forward progress of the caller until <code>DECAY_COPY(std::forward<F>(f))()</code> finishes execution.	<i>Synchronization:</i> The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the
Executor implementations should use the supplied allocator (if any) to allocate			[intro.multithread]) the

any memory required to store the function object. Prior to invoking the invocation of `f`.
function object, the executor shall deallocate any memory allocated. [Note:
Executors defined in this Technical Specification always use the supplied
allocator unless otherwise specified. --end note]

2.1.7 NonBlockingOneWayExecutor requirements

The `NonBlockingOneWayExecutor` requirements add one-way operations that are guaranteed not to block the caller pending completion of submitted function objects.

A type `X` satisfies the `NonBlockingOneWayExecutor` requirements if it satisfies the `HostBasedOneWayExecutor` requirements, as well as the additional requirements listed below.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

In the Table below, `x` denotes a (possibly `const`) value of type `X`, `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements, and `a` denotes a (possibly `const`) value of type `A` satisfying the `ProtoAllocator` requirements.

(Non-blocking one-way executor requirements)

Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
		Creates a parallel execution agent which invokes <code>DECAY_COPY(std::forward<F>(f))()</code> at most once, with the call to <code>DECAY_COPY</code> being evaluated in the thread that called <code>post</code> or <code>defer</code> .	<i>Synchronization:</i> The invocation of <code>post</code> or <code>defer</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f</code> .
<code>x.post(f)</code> <code>x.post(f,a)</code> <code>x.defer(f)</code> <code>x.defer(f,a)</code>		Shall not block forward progress of the caller pending completion of <code>DECAY_COPY(std::forward<F>(f))()</code> . Executor implementations should use the supplied allocator (if any) to allocate any memory required to store the function object. Prior to invoking the function object, the executor shall deallocate any memory allocated. [Note: Executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. --end note]	<i>Note:</i> Although the requirements placed on <code>defer</code> are identical to <code>post</code> , the use of <code>post</code> conveys a preference that the caller does not block the first step of <code>f</code> 's progress, whereas <code>defer</code> conveys a preference that the caller does block the first step of <code>f</code> . One use of <code>defer</code> is to convey the intention of the caller that <code>f</code> is a continuation of the current call context. The executor may use this information to optimize or otherwise adjust the way in which <code>f</code> is invoked.

2.1.8 TwoWayExecutor requirements

The `TwoWayExecutor` requirements form the basis of the two-way executor concept taxonomy; every two-way executor satisfies the `TwoWayExecutor` requirements. This set of requirements specifies operations for creating execution agents that synchronize with the thread which created them.

In the Table below, `f`, denotes a `MoveConstructible` function object with zero arguments whose result type is `R`, and `x` denotes an object of type `X`.

A type `X` satisfies the `TwoWayExecutor` requirements if: * `X` satisfies the `BaseExecutor` requirements. * For any `f` and `x`, the expressions in Table are valid and have the indicated semantics.

(Two-Way Executor requirements)

Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
<code>x.async_execute(std::move(f))</code>	A type that satisfies the <code>Future</code> requirements for the value type <code>R</code> .	Creates an execution agent which invokes <code>f()</code> . Returns the result of <code>f()</code> via the resulting future object. Returns any exception thrown by <code>f()</code> via	

`x.sync_`
`execute(std::move(f))` ^R

the resulting future object.
May block forward progress of the caller pending completion of `f()`.
Creates an execution agent which invokes `f()`.
Returns the result of `f()`.
Throws any exception thrown by `f()`.

2.1.9 NonBlockingTwoWayExecutor requirements

The `NonBlockingOneWayExecutor` requirements add two-way operations that are guaranteed not to block the caller pending completion of submitted function objects.

In the Table below, `f`, denotes a `MoveConstructible` function object with zero arguments whose result type is `R`, and `x` denotes an object of type `x`.

A type `x` satisfies the `NonBlockingTwoWayExecutor` requirements if:

- `x` satisfies the `TwoWayExecutor` requirements.
- For any `f` and `x`, the expressions in Table are valid and have the indicated semantics.

(Non-Blocking Two-Way Executor requirements)

Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
<code>x.async_</code> <code>post(std::move(f))</code> <code>x.async_</code> <code>defer(std::move(f))</code>	<code>executor_</code> <code>future_t<X,R></code>	Creates an execution agent which invokes <code>f()</code> . Returns the result of <code>f()</code> via the resulting future object. Returns any exception thrown by <code>f()</code> via the resulting future object. Shall not block forward progress of the caller pending completion of <code>f()</code> .	

2.1.10 BulkOneWayExecutor requirements

The `BulkOneWayExecutor` requirements form the basis of the bulk one-way executor concept. This set of requirements specifies operations for creating groups of execution agents in bulk from a single operation which need not synchronize with another thread.

In the Table below,

- `f` denotes a `CopyConstructible` function object with three arguments,
- `n` denotes a shape object whose type is `executor_shape_t<X>`.
- `sf` denotes a `CopyConstructible` function object with one argument whose result type is `s`,
- `i` denotes an object whose type is `executor_index_t<X>`, and
- `s` denotes an object whose type is `s`.

A class `x` satisfies the requirements of a bulk one-way executor if `x` satisfies either the `OneWayExecutor` OR `TwoWayExecutor` requirements and the expressions of Table are valid and have the indicated semantics.

(Bulk one-way executor requirements)

Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
<code>x.bulk_</code> <code>execute(f, n, sf)</code>	<code>void</code>	Creates a group of execution agents of shape <code>n</code> which invoke <code>f(i, s)</code> . This group of execution agents shall fulfill the forward progress requirements of <code>executor_execution_ category_t<X></code>	Effects: invokes <code>sf(n)</code> on an unspecified execution agent.

2.1.11 BulkTwoWayExecutor requirements

The `BulkTwoWayExecutor` requirements form the basis of the bulk two-way executor concept. This set of requirements specifies operations for creating groups of execution agents in bulk from a single operation with the ability to synchronize these groups of agents with another thread.

In the Table below,

- `f` denotes a `CopyConstructible` function object with three arguments,

- `n` denotes a shape object whose type is `executor_shape_t<X>`.
- `rf` denotes a `CopyConstructible` function object with one argument whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with one argument whose result type is `S`,
- `i` denotes an object whose type is `executor_index_t<X>`,
- `r` denotes an object whose type is `R`,
- `s` denotes an object whose type is `S`, and
- `pred` denotes a future object whose result is `pr`.

A class `X` satisfies the requirements of a bulk two-way executor if `X` satisfies either the `OneWayExecutor` OR `TwoWayExecutor` requirements and the expressions of Table are valid and have the indicated semantics.

(Bulk two-way executor requirements)			
Expression	Return Type	Operational semantics	Assertion/note/ pre-/post-condition
<code>x.bulk_sync_execute(f, n, rf, sf)</code>	<code>R</code>	Creates a group of execution agents of shape <code>n</code> which invoke <code>f(i, r, s)</code>. This group of execution agents shall fulfill the forward progress requirements of <code>executor_execution_category_t<X></code>. Returns the result of <code>rf(n)</code>.	Note: blocks the forward progress of the caller until all invocations of <code>f</code> are finished. Effects: invokes <code>rf(n)</code> on an unspecified execution agent. Effects: invokes <code>sf(n)</code> on an unspecified execution agent.
<code>x.bulk_async_execute(f, n, rf, sf)</code>	<code>executor_future_t<X,R></code>	Creates a group of execution agents of shape <code>n</code> which invoke <code>f(i, r, s)</code>. This group of execution agents shall fulfill the forward progress requirements of <code>executor_execution_category_t<X></code>. Asynchronously returns the result of <code>rf(n)</code> via the resulting future object.	Effects: invokes <code>rf(n)</code> on an unspecified execution agent. Effects: invokes <code>sf(n)</code> on an unspecified execution agent.
<code>x.bulk_then_execute(f, n, rf, pred, sf)</code>	<code>executor_future_t<X,R></code>	Creates a group of execution agents of shape <code>n</code> which invoke <code>f(i, r, pr, s)</code> after <code>pred</code> becomes ready. This group of execution agents shall fulfill the forward progress requirements of <code>executor_execution_category_t<X></code>. Asynchronously returns the result of <code>rf(n)</code> via the resulting future.	Effects: invokes <code>rf(n)</code> on an unspecified execution agent. Effects: invokes <code>sf(n)</code> on an unspecified execution agent. If <code>pred</code>'s result type is <code>void</code>, <code>pr</code> is omitted from <code>f</code>'s invocation.

2.1.12 ExecutorWorkTracker requirements

The `ExecutorWorkTracker` requirements defines operations for tracking future work against an executor.

A type `X` meets the `ExecutorWorkTracker` requirements if it satisfies the requirements of `CopyConstructible` (`C++Std` [`copyconstructible`]) and `Destructible` (`C++Std` [`destructible`]), as well as the additional requirements listed below.

No constructor, comparison operator, copy operation, move operation, swap operation, or member functions `on_work_started` and `on_work_finished` on these types shall exit via an exception.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

In the Table below, `x` denotes an object of type `X`,

(Executor Work Tracker requirements)		
Expression	Return Type	Assertion/note/pre-/post-condition
<code>x.on_work_started()</code>	<code>bool</code>	Shall not exit via an exception.
<code>x.on_work_finished()</code>		Shall not exit via an exception. Precondition: A corresponding preceding call to <code>on_work_started</code> that returned <code>true</code> .

2.2 Executor type traits

2.2.1 Determining that an executor satisfies the executor requirements

```
template<class T> struct is_one_way_executor;
template<class T> struct is_host_based_one_way_executor;
```

```
template<class T> struct is_non_blocking_one_way_executor;
template<class T> struct is_bulk_one_way_executor;
template<class T> struct is_two_way_executor;
template<class T> struct is_bulk_two_way_executor;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
template<class T> struct is_one_way_executor	T meets the syntactic requirements for <code>OneWayExecutor</code> .	T is a complete type.
template<class T> struct is_host_based_one_way_executor	T meets the syntactic requirements for <code>HostBasedOneWayExecutor</code> .	T is a complete type.
template<class T> struct is_non_blocking_one_way_executor	T meets the syntactic requirements for <code>NonBlockingOneWayExecutor</code> .	T is a complete type.
template<class T> struct is_bulk_one_way_executor	T meets the syntactic requirements for <code>BulkOneWayExecutor</code> .	T is a complete type.
template<class T> struct is_two_way_executor	T meets the syntactic requirements for <code>TwoWayExecutor</code> .	T is a complete type.
template<class T> struct is_non_blocking_two_way_executor	T meets the syntactic requirements for <code>NonBlockingTwoWayExecutor</code> .	T is a complete type.
template<class T> struct is_bulk_two_way_executor	T meets the syntactic requirements for <code>BulkTwoWayExecutor</code> .	T is a complete type.

2.2.2 Associated execution context type

```
template<class Executor>
struct executor_context
{
    using type = std::decay_t<decltype(declval<const Executor&>().context())>; // TODO check this
};
```

2.2.3 Associated future type

```
template<class Executor, class T>
struct executor_future
{
    using type = see below;
};
```

The type of `executor_future<Executor, T>::type` is determined as follows:

- if `is_two_way_executor<Executor>` is true, `decltype(declval<const Executor&>().async_execute( declval<T(*)()>()))`;
- otherwise, if `is_one_way_executor<Executor>` is true, `std::future<T>`;
- otherwise, the program is ill formed.

[*Note:* The effect of this specification is that all execute functions of an executor that satisfies the `TwoWayExecutor`, `NonBlockingTwoWayExecutor`, Or `BulkTwoWayExecutor` requirements must utilize the same future type, and that this future type is determined by `async_execute`. Programs may specialize this trait for user-defined `Executor` types. --end note]

2.3 Bulk executor traits

2.3.1 Classifying forward progress guarantees of groups of execution agents

```
struct sequenced_execution_tag {};
struct parallel_execution_tag {};
struct unsequenced_execution_tag {};

// TODO a future proposal can define this category
// struct concurrent_execution_tag {};

template<class Executor>
struct executor_execution_category
{
private:
    // exposition only
    template<class T>
```

```

using helper = typename T::execution_category;

public:
    using type = std::experimental::detected_or_t<
        unsequenced_execution_tag, helper, Executor
    >;
};

```

Components which create groups of execution agents may use *execution categories* to communicate the forward progress and ordering guarantees of these execution agents with respect to other agents within the same group.

The meanings and relative "strength" of these categories are to be defined. Most of the wording for `sequenced_execution_tag`, `parallel_execution_tag`, and `unsequenced_execution_tag` can be migrated from S 25.2.3 p2, p3, and p4, respectively.

2.3.2 Associated shape type

```

template<class Executor>
struct executor_shape
{
private:
    // exposition only
    template<class T>
    using helper = typename T::shape_type;

public:
    using type = std::experimental::detected_or_t<
        size_t, helper, Executor
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};

```

2.3.3 Associated index type

```

template<class Executor>
struct executor_index
{
private:
    // exposition only
    template<class T>
    using helper = typename T::index_type;

public:
    using type = std::experimental::detected_or_t<
        executor_shape_t<Executor>, helper, Executor
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "index type must be an integral type");
};

```

2.4 Executor Customization Points

2.4.1 In general

The functions described in this clause are *executor customization points*. Executor customization points provide a uniform interface to all executor types.

2.4.2 Function template `execution::execute()`

```

template<class OneWayExecutor, class Function>
void execute(const OneWayExecutor& exec, Function&& f);

```

Effects: calls `exec.execute(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_one_way_executor_v<OneWayExecutor>` is true.

2.4.3 Function template `execution::post()`

```

template<class NonBlockingOneWayExecutor, class Function>
void post(const NonBlockingOneWayExecutor& exec, Function&& f);

```

Effects: calls `exec.post(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_non_blocking_one_way_executor_v<NonBlockingOneWayExecutor>` is true.

2.4.4 Function template execution: `::defer()`

```
template<class NonBlockingOneWayExecutor, class Function>
    void defer(const NonBlockingOneWayExecutor& exec, Function&& f);
```

Effects: calls `exec.defer(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_non_blocking_one_way_executor_v<NonBlockingOneWayExecutor>` is true.

2.4.5 Function template execution: `::sync_execute()`

```
template<class TwoWayExecutor, class Function>
    result_of_t<decay_t<Function>()>
        sync_execute(const TwoWayExecutor& exec, Function&& f);
```

Returns: `exec.sync_execute(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_two_way_executor_v<TwoWayExecutor>` is true.

```
template<class OneWayExecutor, class Function>
    result_of_t<decay_t<Function>()>
        sync_execute(const OneWayExecutor& exec, Function&& f);
```

Effects: Calls `exec.execute(g)`, where `g` is a function object of unspecified type that performs `DECAY_COPY(std::forward<Function>(f))()` and stores the result in `r`, with the call to `DECAY_COPY()` being evaluated in the thread that called `sync_execute`. Blocks the caller of `sync_execute` until `g` completes.

Returns: `r`.

Synchronization: The invocation of `sync_execute` synchronizes with (1.10) the invocation of `f`.

Throws: Any uncaught exception thrown by `f`.

Remarks: This function shall not participate in overload resolution unless `is_two_way_executor_v<TwoWayExecutor>` is false and `is_one_way_executor_v<OneWayExecutor>` is true.

2.4.6 Function template execution: `::async_execute()`

```
template<class TwoWayExecutor, class Function>
    executor_future_t<TwoWayExecutor, result_of_t<decay_t<Function>()>>
        async_execute(const TwoWayExecutor& exec, Function&& f);
```

Returns: `exec.async_execute(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_two_way_executor_v<TwoWayExecutor>` is true.

```
template<class OneWayExecutor, class Function>
    std::future<decay_t<result_of_t<decay_t<Function>()>>
        async_execute(const OneWayExecutor& exec, Function&& f);
```

Returns: `exec.execute(std::forward<Function>(f))`.

Effects: Creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Calls `exec.execute(g)` where `g` is a function object of unspecified type that performs `DECAY_COPY(std::forward<Function>(f))`, with the call to `DECAY_COPY` being performed in the thread that called `async_execute`. On successful completion of `DECAY_COPY(std::forward<Function>(f))`, the return value of `DECAY_COPY(std::forward<Function>(f))` is atomically stored in the shared state and the shared state is made ready. If `DECAY_COPY(std::forward<Function>(f))` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready.

Returns: An object of type `std::future<result_of_t<decay_t<Function>()>>()` that refers to the shared state created by `async_execute`.

Synchronization:

- the invocation of `async_execute` synchronizes with (1.10) the invocation of `f`.
- the completion of the invocation of `f` is sequenced before (1.10) the shared state is made ready.

Remarks: This function shall not participate in overload resolution unless `is_two_way_executor_v<TwoWayExecutor>` is false and `is_one_way_executor_v<OneWayExecutor>` is true.

2.4.7 Function template execution: `async_post()`

```
template<class NonBlockingTwoWayExecutor, class Function>
    executor_future_t<NonBlockingTwoWayExecutor, result_of_t<decay_t<Function>()>>
        async_post(const NonBlockingTwoWayExecutor& exec, Function&& f);
```

Returns: `exec.async_post(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_non_blocking_two_way_executor_v<NonBlockingTwoWayExecutor>` is true.

```
template<class NonBlockingOneWayExecutor, class Function>
    std::future<decay_t<result_of_t<decay_t<Function>()>>
        async_post(const NonBlockingOneWayExecutor& exec, Function&& f);
```

Returns: `exec.post(std::forward<Function>(f))`.

Effects: Creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Calls `exec.post(g)` where `g` is a function object of unspecified type that performs `DECAY_COPY(std::forward<Function>(f))`, with the call to `DECAY_COPY` being performed in the thread that called `async_post`. On successful completion of `DECAY_COPY(std::forward<Function>(f))`, the return value of `DECAY_COPY(std::forward<Function>(f))` is atomically stored in the shared state and the shared state is made ready. If `DECAY_COPY(std::forward<Function>(f))` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready.

Returns: An object of type `std::future<result_of_t<decay_t<Function>()>>` that refers to the shared state created by `async_post`.

Synchronization:

- the invocation of `async_post` synchronizes with (1.10) the invocation of `f`.
- the completion of the invocation of `f` is sequenced before (1.10) the shared state is made ready.

Remarks: This function shall not participate in overload resolution unless `is_non_blocking_two_way_executor_v<NonBlockingTwoWayExecutor>` is false and `is_non_blocking_one_way_executor_v<NonBlockingOneWayExecutor>` is true.

2.4.8 Function template execution: `async_defer()`

```
template<class NonBlockingTwoWayExecutor, class Function>
    executor_future_t<NonBlockingTwoWayExecutor, result_of_t<decay_t<Function>()>>
        async_defer(const NonBlockingTwoWayExecutor& exec, Function&& f);
```

Returns: `exec.async_defer(std::forward<Function>(f))`.

Remarks: This function shall not participate in overload resolution unless `is_non_blocking_two_way_executor_v<NonBlockingTwoWayExecutor>` is true.

```
template<class NonBlockingOneWayExecutor, class Function>
    std::future<decay_t<result_of_t<decay_t<Function>()>>
        async_defer(const NonBlockingOneWayExecutor& exec, Function&& f);
```

Returns: `exec.defer(std::forward<Function>(f))`.

Effects: Creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Calls `exec.defer(g)` where `g` is a function object of unspecified type that performs `DECAY_COPY(std::forward<Function>(f))`, with the call to `DECAY_COPY` being performed in the thread that called `async_defer`. On successful completion of `DECAY_COPY(std::forward<Function>(f))`, the return value of `DECAY_COPY(std::forward<Function>(f))` is atomically stored in the shared state and the shared state is made ready. If `DECAY_COPY(std::forward<Function>(f))` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready.

Returns: An object of type `std::future<result_of_t<decay_t<Function>()>>` that refers to the shared state created by `async_defer`.

Synchronization:

- the invocation of `async_defer` synchronizes with (1.10) the invocation of `f`.
- the completion of the invocation of `f` is sequenced before (1.10) the shared state is made ready.

Remarks: This function shall not participate in overload resolution unless `is_non_blocking_two_way_executor_v<NonBlockingTwoWayExecutor>` is false and `is_non_blocking_one_way_executor_v<NonBlockingOneWayExecutor>` is true.

2.4.9 Function template execution: :then_execute()

```
template<class TwoWayExecutor, class Function, class Future>
    executor_future_t<TwoWayExecutor, see-below>
        then_execute(const TwoWayExecutor& exec, Function&& f, Future& predecessor);
```

Returns: `exec.then_execute(std::forward<Function>(f), predecessor)`. The return type is `executor_future_t<Executor, result_of_t<decay_t<Function>()>> when predecessor is a void future. Otherwise, the return type is executor_future_t<Executor, result_of_t<decay_t<Function>(T&)>> where T is the result type of the predecessor future.`

Remarks: This function shall not participate in overload resolution unless `is_two_way_executor_v< TwoWayExecutor>` is true.

```
template<class OneWayExecutor, class Function, class Future>
    executor_future_t<OneWayExecutor, see-below>
        then_execute(const OneWayExecutor& exec, Function&& f, Future& predecessor);
```

Returns: `predecessor.then(std::forward<Function>(f))`. The return type is `executor_future_t<Executor, result_of_t<decay_t<Function>()>> when predecessor is a void future. Otherwise, the return type is executor_future_t<Executor, result_of_t<decay_t<Function>(T&)>> where T is the result type of the predecessor future.`

Synchronization:

- the invocation of `then_execute` synchronizes with (1.10) the invocation of `f`.
- the completion of the invocation of `f` is sequenced before (1.10) the shared state is made ready.

Postconditions: If the predecessor future is not a shared future, then `predecessor.valid() == false`.

Remarks: This function shall not participate in overload resolution unless `is_two_way_executor_v< TwoWayExecutor>` is false and `is_one_way_executor_v< OneWayExecutor>` is true.

2.4.10 Function template execution: :bulk_execute()

```
template<class BulkOneWayExecutor, class Function1, class Function2>
    void bulk_execute(const BulkOneWayExecutor& exec, Function1 f,
        executor_shape_t<BulkOneWayExecutor> shape,
        Function2 shared_factory);
```

Returns: `exec.bulk_execute(f, shape, shared_factory)`.

Remarks: This function shall not participate in overload resolution unless `is_bulk_one_way_executor_v< BulkOneWayExecutor>` is true.

```
template<class OneWayExecutor, class Function1, class Function2>
    void bulk_execute(const OneWayExecutor& exec, Function1 f,
        executor_shape_t<OneWayExecutor> shape,
        Function2 shared_factory);
```

Effects: Performs `exec.execute(g)` where `g` is a function object of unspecified type that:

- Calls `shared_factory()` and stores the result of this invocation to some shared state `shared`.
- Using `exec.execute`, submits a new group of function objects of shape `shape`. Each function object calls `f(idx, shared)`, where `idx` is the index of the execution agent, and `shared` is a reference to the shared state.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.

Synchronization: The completion of the function `shared_factory` happens before the creation of the group of function objects.

Remarks: This function shall not participate in overload resolution unless `is_bulk_one_way_executor_v< BulkOneWayExecutor>` is false and `is_one_way_executor_v< OneWayExecutor>` is true.

2.4.11 Function template execution: :bulk_sync_execute()

```
template<class BulkTwoWayExecutor, class Function1, class Function2, class Function3>
    result_of_t<Function2()>
        bulk_sync_execute(const BulkTwoWayExecutor& exec, Function1 f,
            executor_shape_t<BulkTwoWayExecutor> shape,
            Function2 result_factory, Function3 shared_factory);
```

Returns: `exec.bulk_sync_execute(f, shape, result_factory, shared_factory)`.

Remarks: This function shall not participate in overload resolution unless `is_bulk_two_way_executor_v< BulkTwoWayExecutor>` is

true.

```
template<class OneWayExecutor, class Function1, class Function2, class Function3>
result_of_t<Function2()>
    bulk_sync_execute(const OneWayExecutor& exec, Function1 f,
                    executor_shape_t<OneWayExecutor> shape,
                    Function2 result_factory, Function3 shared_factory);
```

Effects: Performs `exec.execute(g)` where `g` is a function object of unspecified type that:

- Calls `result_factory()` and `shared_factory()`, and stores the results of these invocations to some shared state `result` and `shared` respectively.
- Using `exec.execute`, submits a new group of function objects of shape `shape`. Each function object calls `f(idx, result, shared)`, where `idx` is the index of the execution agent, and `result` and `shared` are references to the respective shared state. Any return value of `f` is discarded.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.
- Blocks the caller until all invocations of `f` are complete and the result is ready.

Returns: An object of type `result_of_t<Function2()>` that refers to the result shared state created by this call to `bulk_sync_execute`.

Synchronization: The completion of the functions `result_factory` and `shared_factory` happen before the creation of the group of function objects.

Remarks: This function shall not participate in overload resolution unless `is_bulk_two_way_executor_v< BulkTwoWayExecutor>` is false and `is_one_way_executor_v< OneWayExecutor>` is true.

2.4.12 Function template execution: `:bulk_async_execute()`

```
template<class BulkTwoWayExecutor, class Function1, class Function2, class Function3>
executor_future_t<const BulkTwoWayExecutor, result_of_t<Function2()>>
    bulk_async_execute(const BulkTwoWayExecutor& exec, Function1 f,
                    executor_shape_t<BulkTwoWayExecutor> shape,
                    Function2 result_factory, Function3 shared_factory);
```

Returns: `exec.bulk_async_execute(f, shape, result_factory, shared_factory)`.

Remarks: This function shall not participate in overload resolution unless `is_bulk_two_way_executor_v< BulkTwoWayExecutor>` is true.

```
template<class OneWayExecutor, class Function1, class Function2, class Function3>
executor_future_t<const OneWayExecutor, result_of_t<Function2()>>
    bulk_async_execute(const OneWayExecutor& exec, Function1 f,
                    executor_shape_t<OneWayExecutor> shape,
                    Function2 result_factory, Function3 shared_factory);
```

Effects: Performs `exec.execute(g)` where `g` is a function object of unspecified type that:

- Calls `result_factory()` and `shared_factory()`, and stores the results of these invocations to some shared state `result` and `shared` respectively.
- Using `exec.execute`, submits a new group of function objects of shape `shape`. Each function object calls `f(idx, result, shared)`, where `idx` is the index of the function object, and `result` and `shared` are references to the respective shared state. Any return value of `f` is discarded.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.

Returns: An object of type `executor_future_t<Executor, result_of_t<Function2()>>` that refers to the shared result state created by this call to `bulk_async_execute`.

Synchronization:

- The invocation of `bulk_async_execute` synchronizes with (1.10) the invocations of `f`.
- The completion of the functions `result_factory` and `shared_factory` happen before the creation of the group of function objects.
- The completion of the invocations of `f` are sequenced before (1.10) the result shared state is made ready.

Remarks: This function shall not participate in overload resolution unless `is_bulk_two_way_executor_v< BulkTwoWayExecutor>` is

false and `is_one_way_executor_v< OneWayExecutor>` is true.

2.4.13 Function template execution: `bulk_then_execute()`

```
template<class BulkTwoWayExecutor, class Function1, class Future, class Function2, class Function3>
    executor_future_t<Executor, result_of_t<Function2()>>
        bulk_then_execute(const BulkTwoWayExecutor& exec, Function1 f,
                          executor_shape_t<BulkTwoWayExecutor> shape,
                          Future& predecessor,
                          Function2 result_factory, Function3 shared_factory);
```

Returns: `exec.bulk_then_execute(f, shape, result_factory, shared_factory)`.

Remarks: This function shall not participate in overload resolution unless `is_bulk_two_way_executor_v< BulkTwoWayExecutor>` is true.

```
template<class OneWayExecutor, class Function1, class Future, class Function2, class Function3>
    executor_future_t<OneWayExecutor, result_of_t<Function2()>>
        bulk_then_execute(const OneWayExecutor& exec, Function1 f,
                          executor_shape_t<OneWayExecutor> shape,
                          Future& predecessor,
                          Function2 result_factory, Function3 shared_factory);
```

Effects: Performs `exec.execute(g)` where `g` is a function object of unspecified type that:

- Calls `result_factory()` and `shared_factory()` in an unspecified execution agent. The results of these invocations are stored to shared state.
- Using `exec.execute`, submits a new group of function objects of shape `shape` after `predecessor` becomes ready. Each execution agent calls `f(idx, result, pred, shared)`, where `idx` is the index of the execution agent, `result` is a reference to the result shared state, `pred` is a reference to the predecessor state if it is not void. Otherwise, each execution agent calls `f(idx, result, shared)`. Any return value of `f` is discarded.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.

Returns: An object of type `executor_future_t<Executor, result_of_t<Function2()>` that refers to the shared result state created by this call to `bulk_then_execute`.

Synchronization:

- the invocation of `bulk_then_execute` synchronizes with (1.10) the invocations of `f`.
- the completion of the functions `result_factory` and `shared_factory` happen before the creation of the group of execution agents.
- the completion of the invocations of `f` are sequenced before (1.10) the result shared state is made ready.

Postconditions: If the predecessor future is not a shared future, then `predecessor.valid() == false`.

Remarks: This function shall not participate in overload resolution unless `is_bulk_two_way_executor_v< BulkTwoWayExecutor>` is false and `is_one_way_executor_v< OneWayExecutor>` is true.

2.5 Executor work guard

```
template<class Executor>
class executor_work_guard
{
public:
    // types:

    typedef Executor executor_type;

    // construct / copy / destroy:

    explicit executor_work_guard(const executor_type& ex) noexcept;
    executor_work_guard(const executor_work_guard& other) noexcept;
    executor_work_guard(executor_work_guard&& other) noexcept;

    executor_work_guard& operator=(const executor_work_guard&) = delete;

    ~executor_work_guard();
```

```

// executor work guard observers:

executor_type get_executor() const noexcept;
bool owns_work() const noexcept;

// executor work guard modifiers:

void reset() noexcept;

private:
    Executor ex_; // exposition only
    bool owns_; // exposition only
};

```

2.5.1 Members

```
explicit executor_work_guard(const executor_type& ex) noexcept;
```

Effects: Initializes `ex_` with `ex`, and `owns_` with the result of `ex_.on_work_started()`.

Postconditions: `ex_ == ex`.

```
executor_work_guard(const executor_work_guard& other) noexcept;
```

Effects: Initializes `ex_` with `other.ex_`. If `other.owns_ == true`, initializes `owns_` with the result of `ex_.on_work_started()`; otherwise, sets `owns_` to false.

Postconditions: `ex_ == other.ex_`.

```
executor_work_guard(executor_work_guard&& other) noexcept;
```

Effects: Initializes `ex_` with `std::move(other.ex_)` and `owns_` with `other.owns_`, and sets `other.owns_` to false.

```
~executor_work_guard();
```

Effects: If `owns_` is true, performs `ex_.on_work_finished()`.

```
executor_type get_executor() const noexcept;
```

Returns: `ex_`.

```
bool owns_work() const noexcept;
```

Returns: `owns_`.

```
void reset() noexcept;
```

Effects: If `owns_` is true, performs `ex_.on_work_finished()`.

Postconditions: `owns_ == false`.

2.6 Polymorphic executor wrappers

2.6.1 General requirements on polymorphic executor wrappers

Polymorphic executors defined in this Technical Specification satisfy the `BaseExecutor`, `DefaultConstructible` (C++Std [defaultconstructible]), and `CopyAssignable` (C++Std [copyassignable]) requirements, and are defined as follows.

```

class C
{
public:
    class context_type; // TODO define this

    // construct / copy / destroy:

    C() noexcept;
    C(nullptr_t) noexcept;
    C(const executor& e) noexcept;
    C(executor&& e) noexcept;
    template<class Executor> C(Executor e);
    template<class Executor, class ProtoAllocator>
        C(allocator_arg_t, const ProtoAllocator& a, Executor e);

    C& operator=(const C& e) noexcept;
    C& operator=(C&& e) noexcept;

```

```

C& operator=(nullptr_t) noexcept;
template<class Executor> C& operator=(Executor e);

~C();

// polymorphic executor modifiers:

void swap(C& other) noexcept;
template<class Executor, class ProtoAllocator>
    void assign(Executor e, const ProtoAllocator& a);

// C operations:

context_type context() const noexcept;

// polymorphic executor capacity:

explicit operator bool() const noexcept;

// polymorphic executor target access:

const type_info& target_type() const noexcept;
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
};

// polymorphic executor comparisons:

bool operator==(const C& a, const C& b) noexcept;
bool operator==(const C& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const C& e) noexcept;
bool operator!=(const C& a, const C& b) noexcept;
bool operator!=(const C& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const C& e) noexcept;

// executor specialized algorithms:

void swap(C& a, C& b) noexcept;

// in namespace std:

template<class Allocator>
    struct uses_allocator<C, Allocator>
        : true_type {};

```

[*Note*: To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more executor objects. *—end note*]

The *target* is the executor object that is held by the wrapper.

2.6.1.1 Polymorphic executor constructors

```
C() noexcept;
```

Postconditions: `!*this`.

```
C(nullptr_t) noexcept;
```

Postconditions: `!*this`.

```
C(const C& e) noexcept;
```

Postconditions: `!*this` if `!e`; otherwise, `*this` targets `e.target()` or a copy of `e.target()`.

```
C(C&& e) noexcept;
```

Effects: If `!e`, `*this` has no target; otherwise, moves `e.target()` or move-constructs the target of `e` into the target of `*this`, leaving `e` in a valid state with an unspecified value.

```
template<class Executor> C(Executor e);
```

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

```
template<class Executor, class ProtoAllocator>
    C(allocator_arg_t, const ProtoAllocator& a, Executor e);
```

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

A copy of the allocator argument is used to allocate memory, if necessary, for the internal data structures of the constructed c object.

2.6.1.2 Polymorphic executor assignment

```
C& operator=(const C& e) noexcept;
```

Effects: `C(e).swap(*this).`

Returns: `*this.`

```
C& operator=(C&& e) noexcept;
```

Effects: Replaces the target of `*this` with the target of `e`, leaving `e` in a valid state with an unspecified value.

Returns: `*this.`

```
C& operator=(nullptr_t) noexcept;
```

Effects: `C(nullptr).swap(*this).`

Returns: `*this.`

```
template<class Executor> C& operator=(Executor e);
```

Effects: `C(std::move(e)).swap(*this).`

Returns: `*this.`

2.6.1.3 Polymorphic executor destructor

```
~C();
```

Effects: If `*this != nullptr`, releases shared ownership of, or destroys, the target of `*this`.

2.6.1.4 Polymorphic executor modifiers

```
void swap(C& other) noexcept;
```

Effects: Interchanges the targets of `*this` and `other`.

```
template<class Executor, class ProtoAllocator>
void assign(Executor e, const ProtoAllocator& a);
```

Effects: `C(allocator_arg, a, std::move(e)).swap(*this).`

2.6.1.5 Polymorphic executor operations

```
context_type context() const noexcept;
```

Requires: `*this != nullptr`.

Returns: A polymorphic wrapper for `e.context()`, where `e` is the target object of `*this`.

2.6.1.6 Polymorphic executor capacity

```
explicit operator bool() const noexcept;
```

Returns: `true` if `*this` has a target, otherwise `false`.

2.6.1.7 Polymorphic executor target access

```
const type_info& target_type() const noexcept;
```

Returns: If `*this` has a target of type `T`, `typeid(T)`; otherwise, `typeid(void)`.

```
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
```

Returns: If `target_type() == typeid(Executor)` a pointer to the stored executor target; otherwise a null pointer value.

2.6.1.8 Polymorphic executor comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns:

- true if !a and !b;
- true if a and b share a target;
- true if e and f are the same type and e == f, where e is the target of a and f is the target of b;
- otherwise false.

```
bool operator==(const C& e, nullptr_t) noexcept;
```

```
bool operator==(nullptr_t, const C& e) noexcept;
```

Returns: !e.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: !(a == b).

```
bool operator!=(const C& e, nullptr_t) noexcept;
```

```
bool operator!=(nullptr_t, const C& e) noexcept;
```

Returns: (bool) e.

2.6.1.9 Polymorphic executor specialized algorithms

```
void swap(C& a, C& b) noexcept;
```

Effects: a.swap(b).

2.6.2 Class one_way_executor

Class one_way_executor satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class one_way_executor
{
public:
    // execution agent creation
    template<class Function>
        void execute(Function&& f) const;
};
```

Class one_way_executor satisfies the OneWayExecutor requirements. The target object shall satisfy the OneWayExecutor requirements.

```
template<class Function>
    void execute(Function&& f) const;
```

Let e be the target object of *this. Let fd be the result of DECAY_COPY(std::forward<Function>(f)).

Effects: Performs e.execute(g), where g is a function object of unspecified type that, when called as g(), performs fd().

2.6.3 Class host_based_one_way_executor

Class host_based_one_way_executor satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class host_based_one_way_executor
{
public:
    // execution agent creation
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void execute(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
};
```

Class host_based_one_way_executor satisfies the HostBasedOneWayExecutor requirements. The target object shall satisfy the HostBasedOneWayExecutor requirements.

```
template<class Function, class ProtoAllocator>
    void execute(Function&& f, const ProtoAllocator& a) const;
```

Let e be the target object of *this. Let a1 be the allocator that was specified when the target was set. Let fd be the result of DECAY_COPY(std::forward<Function>(f)).

Effects: Performs `e.execute(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

2.6.4 Class `non_blocking_one_way_executor`

Class `non_blocking_one_way_executor` satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class non_blocking_one_way_executor
{
public:
    // execution agent creation
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void execute(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void post(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
    template<class Function, class ProtoAllocator = std::allocator<void>>
        void defer(Function&& f, const ProtoAllocator& a = ProtoAllocator()) const;
};
```

Class `non_blocking_one_way_executor` satisfies the `NonBlockingOneWayExecutor` requirements. The target object shall satisfy the `NonBlockingOneWayExecutor` requirements.

```
template<class Function, class ProtoAllocator>
    void execute(Function&& f, const ProtoAllocator& a) const;
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.execute(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

```
template<class Function, class ProtoAllocator>
    void post(Function&& f, const ProtoAllocator& a) const;
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.post(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

```
template<class Function, class ProtoAllocator>
    void defer(Function&& f, const ProtoAllocator& a) const;
```

Let `e` be the target object of `*this`. Let `a1` be the allocator that was specified when the target was set. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.defer(g, a1)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`. The allocator `a` is used to allocate any memory required to implement `g`.

2.6.5 Class `two_way_executor`

Class `two_way_executor` satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class two_way_executor
{
public:
    // execution agent creation
    template<class Function>
        result_of_t<decay_t<Function>()>
            sync_execute(Function&& f) const;
    template<class Function>
        std::future<result_of_t<decay_t<Function>()>>
            async_execute(Function&& f) const;
};
```

Class `two_way_executor` satisfies the `TwoWayExecutor` requirements. The target object shall satisfy the `TwoWayExecutor` requirements.

```
template<class Executor, class Function>
    result_of_t<decay_t<Function>()>
        sync_execute(Function&& f);
```

Let *e* be the target object of **this*. Let *fd* be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs *e.execute(g)*, where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*.

Returns: The return value of *fd()*.

```
template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_execute(Function&& f) const;
```

Let *e* be the target object of **this*. Let *fd* be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs *e.async_execute(g)*, where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*.

Returns: A future with an associated shared state that will contain the result of *fd()*. [Note: *e.async_execute(g)* may return any future type that satisfies the Future requirements, and not necessarily `std::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that object's *then()* member function. When invoked, this continuation stores the result in the outer future's associated shared and makes that shared state ready. --end note]

2.6.6 Class `non_blocking_two_way_executor`

Class `non_blocking_two_way_executor` satisfies the general requirements on polymorphic executor wrappers, with the additional definitions below.

```
class non_blocking_two_way_executor
{
public:
    // execution agent creation
    template<class Function>
        result_of_t<decay_t<Function>()>
            sync_execute(Function&& f) const;
    template<class Function>
        std::future<result_of_t<decay_t<Function>()>>
            async_execute(Function&& f) const;
    template<class Function>
        std::future<result_of_t<decay_t<Function>()>>
            async_post(Function&& f) const;
    template<class Function>
        std::future<result_of_t<decay_t<Function>()>>
            async_defer(Function&& f) const;
};
```

Class `non_blocking_two_way_executor` satisfies the `NonBlockingTwoWayExecutor` requirements. The target object shall satisfy the `NonBlockingTwoWayExecutor` requirements.

```
template<class Executor, class Function>
    result_of_t<decay_t<Function>()>
        sync_execute(Function&& f);
```

Let *e* be the target object of **this*. Let *fd* be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs *e.execute(g)*, where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*.

Returns: The return value of *fd()*.

```
template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_execute(Function&& f) const;
```

Let *e* be the target object of **this*. Let *fd* be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs *e.async_execute(g)*, where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*.

Returns: A future with an associated shared state that will contain the result of *fd()*.

```
template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_post(Function&& f) const;
```

Let *e* be the target object of **this*. Let *fd* be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs *e.async_post(g)*, where *g* is a function object of unspecified type that, when called as *g()*, performs *fd()*.

Returns: A future with an associated shared state that will contain the result of *fd()*.

```
template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_defer(Function&& f) const;
```

Let `e` be the target object of `*this`. Let `fd` be the result of `DECAY_COPY(std::forward<Function>(f))`.

Effects: Performs `e.async_defer(g)`, where `g` is a function object of unspecified type that, when called as `g()`, performs `fd()`.

Returns: A future with an associated shared state that will contain the result of `fd()`.

2.7 Thread pool type

XXX Consider whether we should include a wording for a concurrent executor which would satisfy the needs of `async` (thread pool provides parallel execution semantics).

2.7.1 Header `<thread_pool>` synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread_pool;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

2.7.2 Class `thread_pool`

This class represents a statically sized thread pool as a common/basic resource type. This pool provides an effectively unbounded input queue and as such calls to add tasks to a `thread_pool`'s executor will not block on the input queue.

```
class thread_pool
{
public:
    class executor_type;

    // construction/destruction
    thread_pool();
    explicit thread_pool(std::size_t num_threads);

    // nocopy
    thread_pool(const thread_pool&) = delete;
    thread_pool& operator=(const thread_pool&) = delete;

    // stop accepting incoming work and wait for work to drain
    ~thread_pool();

    // attach current thread to the thread pools list of worker threads
    void attach();

    // signal all work to complete
    void stop();

    // wait for all threads in the thread pool to complete
    void wait();

    // placeholder for a general approach to getting executors from
    // standard contexts.
    executor_type executor() noexcept;
};
```

```
bool operator==(const thread_pool& a, const thread_pool& b) noexcept;
bool operator!=(const thread_pool& a, const thread_pool& b) noexcept;
```

The class `thread_pool` satisfies the `ExecutionContext` requirements.

For an object of type `thread_pool`, *outstanding work* is defined as the sum of:

- the total number of calls to the `on_work_started` function that returned `true`, less the total number of calls to the `on_work_finished` function, on any executor associated with the `thread_pool`.
- the number of function objects that have been added to the `thread_pool` via the `thread_pool` executor, but not yet executed; and

- the number of function objects that are currently being executed by the `thread_pool`.

The `thread_pool` member functions `executor`, `attach`, `wait`, and `stop`, and the `thread_pool::executor_type` copy constructors and member functions, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

2.7.2.1 Construction and destruction

```
thread_pool();
```

Effects: Constructs a `thread_pool` object with an implementation defined number of threads of execution, as if by creating objects of type `std::thread`.

```
thread_pool(std::size_t num_threads);
```

Effects: Constructs a `thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`. (QUESTION: Do we want to allow 0?)

```
~thread_pool();
```

Effects: Destroys an object of class `thread_pool`. Performs `stop()` followed by `wait()`.

2.7.2.2 Worker Management

```
void attach();
```

Effects: adds the calling thread to the pool of workers. Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `thread_pool` object construction have completed. (Note: The implementation is encouraged, but not required, to use the attached thread to execute submitted function objects. RATIONALE: implementations in terms of the Windows thread pool cannot utilise user-provided threads. --end note) (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete. Subsequent calls to `attach` complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is 0. Blocks the calling thread (C++Std [defs.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent calls to `attach` complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

2.7.2.3 Executor Creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool.

2.7.2.4 Comparisons

```
bool operator==(const thread_pool& a, const thread_pool& b) noexcept;
```

Returns: `std::addressof(a) == std::addressof(b)`.

```
bool operator!=(const thread_pool& a, const thread_pool& b) noexcept;
```

Returns: `!(a == b)`.

2.7.3 Class `thread_pool::executor_type`

```
class thread_pool::executor_type
{
public:
    // types:
```

```

typedef parallel_execution_category execution_category;
typedef std::size_t shape_type;
typedef std::size_t index_type;

// construct / copy / destroy:

executor_type(const executor_type& other) noexcept;
executor_type(executor_type&& other) noexcept;

executor_type& operator=(const executor_type& other) noexcept;
executor_type& operator=(executor_type&& other) noexcept;

// executor operations:

bool running_in_this_thread() const noexcept;

thread_pool& context() const noexcept;

bool on_work_started() const noexcept;
void on_work_finished() const noexcept;

template<class Func, class ProtoAllocator = std::allocator<void>>
    void execute(Func&& f, const ProtoAllocator& a = ProtoAllocator()) const;

template<class Func, class ProtoAllocator = std::allocator<void>>
    void post(Func&& f, const ProtoAllocator& a = ProtoAllocator()) const;

template<class Func, class ProtoAllocator = std::allocator<void>>
    void defer(Func&& f, const ProtoAllocator& a = ProtoAllocator()) const;

template<class Function>
    result_of_t<decay_t<Function>()>
        sync_execute(Function&& f) const;

template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_execute(Function&& f) const;

template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_post(Function&& f) const;

template<class Function>
    std::future<result_of_t<decay_t<Function>()>>
        async_defer(Function&& f) const;

template<class Function1, class Function2>
    void bulk_execute(Function1 f, shape_type shape,
        Function2 shared_factory) const;

template<class Function1, class Function2, class Function3>
    result_of_t<Function2()>
    bulk_sync_execute(Function1 f, shape_type shape,
        Function2 result_factory,
        Function3 shared_factory) const;

template<class Function1, class Function2, class Function3>
    std::future<result_of_t<Function2()>>
    bulk_async_execute(Function1 f, shape_type shape,
        Function2 result_factory,
        Function3 shared_factory) const;
};

bool operator==(const thread_pool::executor_type& a,
    const thread_pool::executor_type& b) noexcept;
bool operator!=(const thread_pool::executor_type& a,
    const thread_pool::executor_type& b) noexcept;

```

`thread_pool::executor_type` is a type satisfying the `NonBlockingOneWayExecutor`, `NonBlockingTwoWayExecutor`, `BulkOneWayExecutor`, `BulkTwoWayExecutor`, and `ExecutorWorkTracker` requirements. Objects of type `thread_pool::executor_type` are associated with a `thread_pool`, and function objects submitted using the `execute`, `post`, `defer`, `sync_execute`, `async_execute`, `async_post`, `async_defer`, `bulk_execute`, `bulk_sync_execute`, and `bulk_async_execute` member functions will be executed by the `thread_pool`.

2.7.3.1 Constructors

```

executor_type(const executor_type& other) noexcept;

```

Postconditions: *this == other.

```
executor_type(executor_type&& other) noexcept;
```

Postconditions: *this is equal to the prior value of other.

2.7.3.2 Assignment

```
executor_type& operator=(const executor_type& other) noexcept;
```

Postconditions: *this == other.

Returns: *this.

```
executor_type& operator=(executor_type&& other) noexcept;
```

Postconditions: *this is equal to the prior value of other.

Returns: *this.

2.7.3.3 Operations

```
bool running_in_this_thread() const noexcept;
```

Returns: true if the current thread of execution is a thread that was created by or attached to the associated thread_pool object.

```
thread_pool& context() const noexcept;
```

Returns: A reference to the associated thread_pool object.

```
bool on_work_started() const noexcept;
```

Effects: Increments the count of outstanding work associated with the thread_pool.

```
void on_work_finished() const noexcept;
```

Effects: Decrements the count of outstanding work associated with the thread_pool.

```
template<class Func, class ProtoAllocator = std::allocator<void>>
    void execute(Func&& f, const ProtoAllocator& a = ProtoAllocator()) const;
```

Effects: If running_in_this_thread() is true, calls DECAY_COPY(forward<Func>(f))(). *[Note: *If f exits via an exception, the exception propagates to the caller of execute. *--end note]* Otherwise, calls post(forward<Func>(f), a).

```
template<class Func, class ProtoAllocator = std::allocator<void>>
    void post(Func&& f, const ProtoAllocator& a = ProtoAllocator()) const;
```

Effects: Adds f to the thread_pool.

```
template<class Func, class ProtoAllocator = std::allocator<void>>
    void defer(Func&& f, const ProtoAllocator& a = ProtoAllocator()) const;
```

Effects: Adds f to the thread_pool.

```
template<class Function>
    result_of_t<decay_t<Function>()>
    sync_execute(Function&& f) const;
```

Effects: If running_in_this_thread() is true, calls DECAY_COPY(forward<Func>(f))(). Otherwise, adds f to the thread_pool and blocks the caller pending completion of f.

Returns: The return value of f.

Throws: Any uncaught exception thrown by f.

```
template<class Function>
    future<result_of_t<decay_t<Function>()>>
    async_execute(Function&& f) const;
template<class Function>
    future<result_of_t<decay_t<Function>()>>
    async_post(Function&& f) const;
template<class Function>
    future<result_of_t<decay_t<Function>()>>
    async_defer(Function&& f) const;
```

Effects: Creates an asynchronous provider with an associated shared state (C++Std [futures.state]). Adds `f` to the `thread_pool`. On successful completion of `f`, the return value of `f` is atomically stored in the shared state and the shared state is made ready. If `f` exits via an exception, the exception is atomically stored in the shared state and the shared state is made ready.

Returns: An object of type `future<result_of_t<decay_t<Function>>()>` that refers to the shared state created by `async_execute`.

```
template<class Function1, class Function2>
void bulk_execute(Function1 f, shape_type shape,
                 Function2 shared_factory) const;
```

Effects: Submits a function object to the thread pool that:

- Calls `shared_factory()` and stores the result of this invocation to some shared state `shared`.
- Submits a new group of function objects of shape `shape`. Each function object calls `f(idx, shared)`, where `idx` is the index of the execution agent, and `shared` is a reference to the shared state.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.

Synchronization: The completion of the function `shared_factory` happens before the creation of the group of function objects.

```
template<class Function1, class Function2, class Function3>
result_of_t<Function2()>
bulk_sync_execute(Function1 f, shape_type shape,
                 Function2 result_factory,
                 Function3 shared_factory) const;
```

Effects: Submits a function object to the thread pool that:

- Calls `result_factory()` and `shared_factory()`, and stores the results of these invocations to some shared state `result` and `shared` respectively.
- Submits a new group of function objects of shape `shape`. Each function object calls `f(idx, result, shared)`, where `idx` is the index of the execution agent, and `result` and `shared` are references to the respective shared state. Any return value of `f` is discarded.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.
- Blocks the caller until all invocations of `f` are complete and the result is ready.

Returns: An object of type `result_of_t<Function2()>` that refers to the result shared state created by this call to `bulk_sync_execute`.

Synchronization: The completion of the functions `result_factory` and `shared_factory` happen before the creation of the group of function objects.

```
template<class Function1, class Function2, class Function3>
std::future<result_of_t<Function2()>>
bulk_async_execute(Function1 f, shape_type shape,
                 Function2 result_factory,
                 Function3 shared_factory) const;
```

Effects: Submits a function object to the thread pool that:

- Calls `result_factory()` and `shared_factory()`, and stores the results of these invocations to some shared state `result` and `shared` respectively.
- Submits a new group of function objects of shape `shape`. Each function object calls `f(idx, result, shared)`, where `idx` is the index of the function object, and `result` and `shared` are references to the respective shared state. Any return value of `f` is discarded.
- If any invocation of `f` exits via an uncaught exception, `terminate` is called.

Returns: An object of type `std::future<result_of_t<Function2()>>` that refers to the shared result state created by this call to `bulk_async_execute`.

Synchronization:

- The invocation of `bulk_async_execute` synchronizes with (1.10) the invocations of `f`.
- The completion of the functions `result_factory` and `shared_factory` happen before the creation of the group of function objects.

- The completion of the invocations of `f` are sequenced before (1.10) the result shared state is made ready.

2.7.3.4 Comparisons

```
bool operator==(const thread_pool::executor_type& a,
               const thread_pool::executor_type& b) noexcept;
```

Returns: `a.context() == b.context()`.

```
bool operator!=(const thread_pool::executor_type& a,
               const thread_pool::executor_type& b) noexcept;
```

Returns: `!(a == b)`.

2.8 Interoperation with existing facilities

2.8.1 Execution policy interoperation

```
class parallel_execution_policy
{
public:
    // types:
    using execution_category = parallel_execution_tag;
    using executor_type = implementation-defined;

    // executor access
    const executor_type& executor() const noexcept;

    // execution policy factory
    template<class Executor>
    see-below on(Executor&& exec) const;
};

class sequenced_execution_tag { by-analogy-to-parallel_execution_policy };
class parallel_unsequenced_execution_tag { by-analogy-to-parallel_execution_policy };
```

2.8.1.1 Associated executor

Each execution policy is associated with an executor, and this executor is called its *associated executor*.

The type of an execution policy's associated executor shall satisfy the requirements of `BulkTwoWayExecutor`.

When an execution policy is used as a parameter to a parallel algorithm, the execution agents that invoke element access functions are created by the execution policy's associated executor.

The type of an execution policy's associated executor is the same as the member type `executor_type`.

2.8.1.2 Execution category

Each execution policy is categorized by an *execution category*.

When an execution policy is used as a parameter to a parallel algorithm, the execution agents it creates are guaranteed to make forward progress and execute invocations of element access functions as ordered by its execution category.

An execution policy's execution category is given by the member type `execution_category`.

The execution category of an execution policy's associated executor shall not be weaker than the execution policy's execution category.

2.8.1.3 Associated executor access

```
const executor_type& executor() const noexcept;
```

Returns: The execution policy's associated executor.

2.8.1.4 Execution policy factory

```
template<class Executor>
see-below on(Executor&& exec) const;
```

Let `T` be `decay_t<Executor>`.

Returns: An execution policy whose execution category is `execution_category`. If T satisfies the requirements of `BulkTwoWayExecutor`, the returned execution policy's associated executor is equal to `exec`. Otherwise, the returned execution policy's associated executor is an adaptation of `exec`.

XXX TODO: need to define what adaptation means

Remarks: This member function shall not participate in overload resolution unless `is_executor_v<T>` is true and `executor_execution_category_t<T>` is as strong as `execution_category`.

2.8.2 Control structure interoperation

2.8.2.1 Function template `async`

The function template `async` provides a mechanism to invoke a function in a new execution agent created by an executor and provides the result of the function in the future object with which it shares a state.

```
template<class Executor, class Function, class... Args>
executor_future_t<Executor, result_of_t<decay_t<Function>(decay_t<Args>...)>>
async(const Executor& exec, Function&& f, Args&&... args);
```

Returns: Equivalent to:

```
return execution::async_post(exec, [=]{ return INVOKE(f, args...); });
```

XXX This forwarding doesn't look correct to me

2.8.2.2 `std::future::then()`

The member function template `then` provides a mechanism for attaching a *continuation* to a `std::future` object, which will be executed on a new execution agent created by an executor.

```
template<class T>
template<class Executor, class Function>
executor_future_t<Executor, see-below>
future<T>::then(const Executor& exec, Function&& f);
```

2. TODO: Concrete specification

The general idea of this overload of `.then()` is that it accepts a particular type of `OneWayExecutor` that cannot block in `.execute()`. `.then()` stores `f` as the next continuation in the future state, and when the future is ready, creates an execution agent using a copy of `exec`.

One approach is for `.then()` to require a `NonBlockingOneWayExecutor`, and to specify that `.then()` submits the continuation using `exec.post()` if the future is already ready at the time when `.then()` is called, and to submit using `exec.execute()` otherwise.

2.8.2.3 `std::shared_future::then()`

The member function template `then` provides a mechanism for attaching a *continuation* to a `std::shared_future` object, which will be executed on a new execution agent created by an executor.

```
template<class T>
template<class Executor, class Function>
executor_future_t<Executor, see-below>
shared_future<T>::then(const Executor& exec, Function&& f);
```

TODO: Concrete specification

The general idea of this overload of `.then()` is that it accepts a particular type of `OneWayExecutor` that cannot block in `.execute()`. `.then()` stores `f` as the next continuation in the underlying future state, and when the underlying future is ready, creates an execution agent using a copy of `exec`.

One approach is for `.then()` to require a `NonBlockingOneWayExecutor`, and to specify that `.then()` submits the continuation using `exec.post()` if the future is already ready at the time when `.then()` is called, and to submit using `exec.execute()` otherwise.

2.8.2.4 Function template `invoke`

The function template `invoke` provides a mechanism to invoke a function in a new execution agent created by an executor and return result of the function.

```
template<class Executor, class Function, class... Args>
```

```
result_of_t<F&&(Args&&...)>
invoke(const Executor& exec, Function&& f, Args&&... args);
```

Returns: Equivalent to:

```
return execution::sync_execute(exec, [&]{ return INVOKE(f, args...); });
```

2.8.2.5 Task block

2.8.2.5.1 Function template define_task_block_restore_thread()

```
template<class Executor, class F>
void define_task_block_restore_thread(const Executor& exec, F&& f);
```

Requires: Given an lvalue tb of type task_block, the expression f(tb) shall be well-formed.

Effects: Constructs a task_block tb, creates a new execution agent, and calls f(tb) on that execution agent.

Throws: exception_list, as specified in version two of the Paralellism TS.

Postconditions: All tasks spawned from f have finished execution.

Remarks: Unlike define_task_block, define_task_block_restore_thread always returns on the same thread as the one on which it was called.

2.8.2.5.2 task_block member function template run

```
template<class Executor, class F>
void run(const Executor& exec, F&& f);
```

Requires: F shall be MoveConstructible. DECAY_COPY(std::forward<F>(f))() shall be a valid expression.

Preconditions: *this shall be an active task_block.

Effects: Evaluates DECAY_COPY(std::forward<F>(f))(), where DECAY_COPY(std::forward<F>(f)) is evaluated synchronously within the current thread. The call to the resulting copy of the function object is permitted to run on an execution agent created by exec in an unordered fashion relative to the sequence of operations following the call to run(exec, f) (the continuation), or indeterminately-sequenced within the same thread as the continuation. The call to run synchronizes with the next invocation of wait on the same task_block or completion of the nearest enclosing task_block (i.e., the define_task_block or define_task_block_restore_thread that created this task_block.

Throws: task_cancelled_exception, as described in version 2 of the Parallelism TS.

3 Relationship to other proposals and specifications

3.1 Networking TS

Executors in the Networking TS may be defined as refinements of the type requirements in this proposal, as illustrated below. In addition to these requirements, some minor changes would be required to member function names and parameters used in the Networking TS, to conform to the requirements defined in this proposal.

3.1.1 NetworkingExecutor requirements

A type X satisfies the NetworkingExecutor requirements if it satisfies the NonBlockingOneWayExecutor requirements, the ExecutorWorkTracker requirements, and satisfies the additional requirements listed below.

In the Table below, x denotes a (possibly const) value of type x.

expression	return type	assertion/note	pre/post-condition
x.context()	net::execution_context&, or E& where E is a type that satisfies the NetworkingExecutionContext requirements.		

3.1.2 NetworkingExecutionContext requirements

A type X satisfies the NetworkingExecutionContext requirements if it satisfies the ExecutionContext requirements, is publicly and

unambiguously derived from `net::execution_context`, and satisfies the additional requirements listed below.

In the Table below, `x` denotes a value of type `x`.

(NetworkingExecutionContext requirements)		
expression	return type	assertion/note pre/post-condition
	type meeting	
<code>X::executor_type</code>	<code>NetworkingExecutor</code>	
	requirements	
<code>x.~X()</code>		Destroys all unexecuted function objects that were submitted via an executor object that is associated with the execution context.
<code>x.get_executor()</code>	<code>X::executor_type</code>	Returns an executor object that is associated with the execution context.

4 Future work